AI-Native Full-Stack Developer

Interview & Role Revision Guide — PHP / JavaScript / Docker

PREPARED FOR
Interview Preparation

BY
Chris Garlick

DATE
2026-07-14

About This Guide

Focused revision for the AI-Native Full-Stack Developer role — a small, fast-moving, remote team where you build and maintain PHP (8.x) web apps while wiring in LLM APIs, agents, and automation pipelines. The role sits at the intersection of software engineering, AI tooling, and creative problem-solving. Salary £45,000, mid-level (2–5 yrs), reports to Head of Technology / CTO.

Each section opens with why it matters for this role, gives the concept in plain English, then a PHP/JS example or checklist you can speak to. The last pages give you likely interview questions and smart questions to ask them.

CORE ENGINEERING

1. The Role, Mapped
2. PHP 8.x & OOP
3. Working with Legacy PHP
4. Design Patterns
5. A Major Framework (Laravel)
6. REST API Design
7. Auth: OAuth & JWT
8. Web Security
9. Databases (MySQL / PG)

AI & AUTOMATION

10. LLM API Integration
11. Prompt Engineering
12. Loop Engineering & Agents
13. Model Orchestration
14. Coding Agents (Claude Code etc.)
15. Automation Pipelines
16. Webhooks & Idempotency

STACK & DELIVERY

17. JavaScript / TypeScript
18. React & Vue
19. Docker & Containers
20. CI/CD (GitLab)
21. Cloud Deployment
22. Testing
23. Desirable: CMS & E-commerce
24. The Ideal Candidate
25. Interview Q&A
26. Questions to Ask Them

How to use it: read every Why it matters box, rehearse the Say this lines out loud, and be ready to tie each topic back to real products you've shipped — they explicitly want LLM integration in real products, not just side projects. Show AI is a core part of your toolkit, not an add-on.

1 · The Role, Mapped to You

WHY IT MATTERS

This is not a traditional backend role. They want someone equally comfortable prompting an LLM and writing PHP. Map every requirement to a concrete thing you've built — specifics beat vague claims. They're lean and fast-moving, so signal that you ship, own features end-to-end, and self-direct.

Their requirements → your evidence

They want	What to say / show
Strong PHP (OOP, patterns, framework)	A PHP/Laravel app you built — models, migrations, services, tests.
LLM APIs in real products	A feature where you called OpenAI/Anthropic/Gemini in production.
Automation platform / pipeline	A workflow you automated — cron, queue, CI, or an agent loop.
Cross-stack (JS/TS, React/Vue)	A frontend you built that talks to your PHP backend.
REST, OAuth/JWT, security	An authenticated API you designed; how you handled tokens.
Git, CI/CD, cloud deploys	A GitLab/GitHub pipeline that deploys to Docker/cloud.
Docker	A Dockerfile / compose stack you wrote and shipped.
Coding agents (Claude Code, Codex)	How you actually use agents day-to-day to move faster.

THE TELL

They value curiosity about fast-moving AI tooling and people who proactively spot where to apply it. Come with a recent example: a tool you tried last month, what you built with it, what you'd bring to their stack.

SAY THIS

"I treat the LLM as part of the stack — I'll prototype with an agent, wire the API into the product, then harden it with tests, retries, and cost controls. I like owning a feature from scoping to monitoring."

2 · PHP 8.x & OOP

WHY IT MATTERS

The role asks for strong, demonstrable PHP including OOP. PHP 8 added real language power — be fluent in the modern syntax, not PHP 5 habits.

PHP 8.x features to name-drop

Feature	One-liner
Constructor promotion	Declare + assign properties in the constructor signature.
Enums	First-class enums (backed or pure) replace class constants.
Readonly properties	Immutable once set — great for value objects / DTOs.
Named arguments	Skip optional params by name; self-documenting calls.
Match expression	Strict (===) switch that returns a value; no fall-through.
Nullsafe operator	`$a?->b()?->c` short-circuits on null.
Union / intersection types	`int|string`, `Countable&Iterator` in signatures.
First-class callable	`$fn = strlen(...);` — clean callable references.
Fibers / attributes	Coroutine primitive; attributes power modern frameworks.

```
enum Status: string {
    case Draft = 'draft';
    case Published = 'published';

    public function label(): string {
        return match ($this) {
            Status::Draft     => 'Draft',
            Status::Published => 'Published',
        };
    }
}

final class Article {
    public function __construct(
        public readonly string $title,
        public readonly Status $status = Status::Draft,
    ) {}
}

$a = new Article(title: 'AI-Native PHP', status: Status::Published);
echo $a->status->label(); // Published
```

Four OOP pillars, fast: Encapsulation (hide state behind methods), Abstraction (interfaces / abstract classes), Inheritance (extend — but prefer composition), Polymorphism (same interface, different behaviour). Be ready to say “I favour composition over inheritance and program to interfaces.”

3 · Working with Legacy PHP (5.x → 8)

WHY IT MATTERS

This team has legacy PHP, some still on PHP 5. A real part of the job is maintaining and modernising it without breaking what works. PHP 5 has been end-of-life since Dec 2018 / Jan 2019 — no security patches — so getting off it is both a quality and a security priority. Show you can be pragmatic: keep the lights on, then improve incrementally.

What changed since PHP 5 (the gotchas)

Legacy PHP 5 habit	Modern reality
mysql_* functions	Removed in PHP 7 — use PDO/mysqli with prepared statements.
No scalar / return type hints	PHP 7+ adds scalar, return, nullable & union types — add gradually.
No namespaces / manual require	Composer + PSR-4 autoloading; drop the require_once spaghetti.
each(), create_function(), split()	Removed/deprecated — use foreach, closures, preg_split.
Fatal errors everywhere	PHP 7 turns many fatals into catchable \Error / \Throwable.
Loose == comparisons	PHP 8 fixed string↔number juggling — old code may change behaviour.
register_globals / magic_quotes	Long gone — and a classic source of the app's old security holes.
Mixed HTML + PHP in one file	Separate concerns; move logic behind classes/services.

```
// Legacy PHP 5 — removed in PHP 7, and SQL-injectable
$id  = $_GET['id'];
$res = mysql_query("SELECT * FROM users WHERE id = $id");
$row = mysql_fetch_assoc($res);

// Modern equivalent — PDO + prepared statement
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = :id');
$stmt->execute(['id' => (int) $_GET['id']]);
$row = $stmt->fetch(PDO::FETCH_ASSOC);
```

How to modernise safely

Step	Why
Safety net first	Add characterisation tests so you can change code without changing behaviour.
Upgrade incrementally	Go 5.6 → 7.4 → 8.x one hop at a time; get green before the next bump.
Automate the grunt work	Rector applies mechanical upgrades; PHPCompatibility (phpcs) flags what breaks.
Add static analysis	PHPStan / Psalm surface hidden type bugs the old runtime hid.

Step	Why
Strangler-fig pattern	Route new features to new code; replace legacy piece by piece, not big-bang.
Isolate what you can't fix yet	Contain EOL code behind a WAF / network boundary; scan dependencies.

Tools to name-drop

Rector (automated refactors/upgrades) · PHPCompatibility sniffs for phpcs · PHPStan / Psalm (static analysis) · PHPUnit / Pest (safety net) · Composer (autoloading & deps). An LLM/coding agent is genuinely useful here for grinding through repetitive migrations — a nice tie-in to the AI-native angle.

SAY THIS

"I'd stabilise the PHP 5 code with characterisation tests, then upgrade in small hops with Rector and PHPStan guarding each step — strangler-fig style, so new features go on modern code while I retire the legacy safely. Getting off an EOL runtime is a security win too."

4 · Design Patterns

WHY IT MATTERS

"Design patterns" is called out explicitly. You don't need all 23 GoF patterns — you need the handful you actually use and can explain why.

Pattern	When you'd reach for it
Dependency Injection	Pass collaborators in, don't `new` them inside — testable, swappable.
Repository	Wrap data access behind an interface; swap DB/cache/API freely.
Strategy	Pick an algorithm at runtime — e.g. choose which LLM provider to call.
Factory	Centralise object creation when construction is non-trivial.
Adapter	Wrap a 3rd-party SDK (OpenAI/Anthropic) behind your own interface.
Decorator	Add behaviour (caching, logging, retries) without touching the core.
Observer / Events	React to something happening — dispatch events, run listeners.
Singleton	One shared instance (used sparingly — DI container usually better).

```
interface LlmClient {
    public function complete(string $prompt): string;
}

final class AnthropicClient implements LlmClient { /* ... */
    public function complete(string $prompt): string { /* call API */
return ''; }
}
final class OpenAiClient implements LlmClient { /* ... */
    public function complete(string $prompt): string { /* call API */
return ''; }
}

// Strategy + DI: the caller doesn't care which provider it is.
final class Summariser {
    public function __construct(private LlmClient $llm) {}
    public function summarise(string $text): string {
        return $this->llm->complete("Summarise:\n\n{$text}");
    }
}
```

SOLID, in a breath

Single responsibility · Open/closed · Liskov substitution · Interface segregation · Dependency inversion. The Adapter above is the D — depend on `LlmClient`, not a vendor SDK.

SAY THIS

"I wrap each LLM vendor behind one interface, so swapping OpenAI for Anthropic — or A/B-testing them — is a one-line change and every call site stays untouched."

5 · A Major Framework (Laravel)

WHY IT MATTERS

They want a major framework. Laravel and Symfony are the big two. Speak to whichever you know deeply — below is Laravel-flavoured, but the concepts (routing, ORM, DI container, migrations, queues) map to Symfony too.

Concept	What to know
Routing & Middleware	Map URLs → controllers; middleware for auth, rate-limit, CORS.
Service Container	IoC container auto-resolves dependencies via type-hints.
Eloquent ORM	Active-record models, relationships, eager loading (avoid N+1).
Migrations & Seeders	Version-controlled schema; reproducible DB state.
Validation & Form Requests	Validate input at the edge before it hits your logic.
Queues & Jobs	Push slow work (LLM calls, emails) off the request cycle.
Events & Listeners	Decouple side-effects from the action that triggers them.
Artisan & Scheduler	CLI commands + cron-driven task scheduling.

```
// A queued job that calls an LLM off the request cycle.
class GenerateSummary implements ShouldQueue {
    public function __construct(public int $articleId) {}

    public function handle(Summariser $summariser): void {
        $article = Article::findOrFail($this->articleId);
        $article->update([
            'summary' => $summariser->summarise($article->body),
        ]);
    }
}

// Dispatch it from a controller — returns instantly, work runs in
background.
GenerateSummary::dispatch($article->id);
```

N+1 killer: `Article::with('author')->get()` eager-loads authors in 2 queries instead of N+1. Be ready to say you profile with Laravel Telescope / Debugbar and add DB indexes on foreign keys and frequently-filtered columns.

6 · REST API Design

WHY IT MATTERS

Solid understanding of REST API design is a named requirement — and everything AI-adjacent (LLM calls, webhooks, SaaS integrations) is API work.

Principle	In practice
Resource-oriented URLs	`/articles`, `/articles/42` — nouns, not verbs.
HTTP verbs	GET read · POST create · PUT/PATCH update · DELETE remove.
Status codes	200/201, 204, 400, 401, 403, 404, 409, 422, 429, 500.
Statelessness	Each request self-contained; auth token on every call.
Versioning	`/v1/...` or header — never break existing clients.
Pagination & filtering	`?page=&per_page=` / cursor; return links + total.
Idempotency	PUT/DELETE idempotent; use idempotency keys on POST.
Consistent errors	One error shape: `{ error, status, details }`.

```
// A predictable, versioned response envelope
{
  "data": [
    { "id": 42, "title": "AI-Native PHP", "status": "published" }
  ],
  "meta": { "page": 1, "per_page": 20, "total": 137 },
  "links": { "next": "/v1/articles?page=2" }
}
```

422 vs 400: use 422 Unprocessable Entity for validation failures (well-formed but semantically wrong), 400 for malformed requests. 429 for rate limits with a `Retry-After` header. Knowing these distinctions signals real API experience.

7 · Authentication — OAuth & JWT

WHY IT MATTERS

The JD names OAuth and JWT directly. Know what each is, when to use it, and the failure modes.

OAuth 2.0

Delegated authorization — lets a user grant your app scoped access to a third-party account without sharing their password. Flows: Authorization Code + PKCE (web/mobile apps), Client Credentials (server-to-server). You get an access token (+ refresh token). This is how you'd integrate a SaaS platform's API on a user's behalf.

JWT

A JSON Web Token — a signed, self-contained token: `header.payload.signature`, base64url-encoded. Stateless: the server verifies the signature instead of a DB session lookup. Store claims (`sub`, `exp`, `roles`) in the payload — never secrets (it's readable, only tamper-proof). Keep expiry short; use refresh tokens.

Question they may ask	Crisp answer
Session vs JWT?	Sessions = server state, easy revoke. JWT = stateless, scales, harder to revoke.
How to revoke a JWT?	Short expiry + refresh tokens; or a denylist / token version claim.
Where to store the token?	HttpOnly, Secure cookie beats localStorage (XSS-safe).
Authn vs Authz?	Authentication = who you are. Authorization = what you may do.
OAuth = login?	Not quite — that's OpenID Connect (an identity layer on OAuth 2).

8 · Web Security Fundamentals

WHY IT MATTERS

Web security fundamentals are required. Speak the OWASP language and show it's baked into how you build — plus the AI-specific risks, which will impress on this role.

Risk	Defence
SQL Injection	Prepared statements / parameterised queries — never string-concat SQL.
XSS	Escape output; CSP header; sanitise HTML; framework auto-escaping.
CSRF	Anti-CSRF tokens; SameSite cookies.
Broken auth	Hash passwords (bcrypt/argon2), rate-limit logins, MFA.
Sensitive data	TLS everywhere; encrypt at rest; secrets in a vault, not code.
SSRF	Allowlist outbound hosts — key when your server calls user-supplied URLs.
Prompt injection	Treat LLM input/output as untrusted; sandbox tool calls; validate.
Secret leakage to LLMs	Never send secrets/PII to a model without need; redact & log usage.

AI-native angle: prompt injection is the new injection class — a malicious document can hijack your agent's instructions. Defences: separate system vs user content, least-privilege tool access, human-in-the-loop for destructive actions, and validate/parse model output before acting on it. Mentioning this unprompted shows you think about AI security, not just AI features.

9 · Databases (MySQL / PostgreSQL)

WHY IT MATTERS

Manage relational databases (MySQL / PostgreSQL) is a core responsibility. Know schema design, indexing, and how to diagnose a slow query.

Topic	What to say
Normalisation	3NF to avoid duplication; denormalise deliberately for read speed.
Indexes	Index FKs + WHERE/ORDER-BY columns; composite indexes are ordered.
EXPLAIN	Read the query plan; watch for full table scans & filesorts.
Transactions & ACID	Wrap multi-step writes; understand isolation levels & deadlocks.
N+1	Eager-load relations; batch queries.
MySQL vs Postgres	PG: richer types (JSONB, arrays), stricter, CTEs; MySQL: ubiquitous, fast reads.
Migrations	Version schema in code; make migrations reversible & zero-downtime.

```
-- Diagnose before you optimise
EXPLAIN ANALYZE
SELECT a.id, a.title
FROM articles a
WHERE a.status = 'published'
ORDER BY a.published_at DESC
LIMIT 20;

-- Composite index that serves both the filter and the sort
CREATE INDEX idx_articles_status_published
    ON articles (status, published_at DESC);
```

Postgres JSONB is worth a mention on an AI role — great for storing variable-shape LLM output or embeddings metadata, and with pgvector you can do similarity search for RAG directly in Postgres.

10 · LLM API Integration

WHY IT MATTERS

The headline of the role: Integrate LLM APIs (OpenAI, Anthropic, Gemini) into products and internal tooling. They want real production integration — retries, cost control, streaming, structured output — not a demo.

Concern	How you handle it
Auth & secrets	API key in env/vault; never in the client bundle.
Latency	Stream tokens (SSE); queue non-interactive calls.
Reliability	Retry with exponential backoff on 429/5xx; timeouts.
Cost	Cache responses; pick the right model per task; cap max tokens.
Structured output	Ask for JSON / use tool-calling; validate the schema.
Context limits	Chunk & summarise; RAG for large corpora.
Observability	Log prompts, tokens, latency, cost per call.
Provider abstraction	One interface, swap vendors (see §3 Adapter/Strategy).

```
// Anthropic Messages API call in PHP (Guzzle), with structured output
$response = $http->post('https://api.anthropic.com/v1/messages', [
    'headers' => [
        'x-api-key'         => env('ANTHROPIC_API_KEY'),
        'anthropic-version' => '2023-06-01',
    ],
    'json' => [
        'model'      => 'claude-opus-4-8',
        'max_tokens' => 1024,
        'system'     => 'Reply ONLY with valid JSON matching the
schema.',
        'messages'   => [
            ['role' => 'user', 'content' => "Extract name & sentiment:
{$text}"],
        ],
    ],
    'timeout' => 30,
]);

$json = json_decode((string) $response->getBody(), true);
$data = json_decode($json['content'][0]['text'], true); // validate
before use
```

RAG in one line: embed your docs → store vectors → retrieve the top-k relevant chunks → stuff them into the prompt → the model answers grounded in your data. Be ready to explain it — it's the most common production LLM pattern.

11 · Prompt Engineering

WHY IT MATTERS

Listed under Desirable Skills (prompt engineering methodologies) and central to the AI-native philosophy. Show you have a method, not just vibes.

Technique	What it does
Clear role + task	Set a system role, then a specific, bounded task.
Few-shot examples	Show 2–3 input⟶output pairs to lock the format.
Chain-of-thought	Ask the model to reason step-by-step for hard tasks.
Structured output	Demand JSON / a schema; validate and retry on failure.
Delimiters	Fence user content so it can't be mistaken for instructions.
Grounding (RAG)	Give the facts in-context; tell it to only use those.
Eval loop	Build a test set; measure prompt changes, don't guess.

Anti-patterns

Vague asks · one giant mega-prompt · trusting output without validation · no evals · re-sending huge context every call (cache it) · putting instructions after untrusted user text.

SAY THIS

"I treat prompts like code — version-controlled, with a small eval set so I can measure whether a change actually improved accuracy before it ships."

12 · Loop Engineering & Agents

WHY IT MATTERS

Named verbatim: Build and maintain AI-powered workflows — loop engineering, agents, and model orchestration. This is the frontier bit; being able to explain it plainly sets you apart.

An agent = an LLM in a loop with tools. It observes, decides, calls a tool, sees the result, and repeats until the task is done. Loop engineering is designing that loop well: what context goes in, when to stop, how to recover from errors, and how to keep it from running away.

```
// The agent loop, distilled
$messages = [['role' => 'user', 'content' => $task]];

for ($step = 0; $step < MAX_STEPS; $step++) {
    $reply = $llm->respond($messages, tools: $tools);

    if ($reply->isFinalAnswer()) {
        return $reply->text;                // stop condition
    }

    $result = $toolRegistry->run(           // execute the requested
tool
        $reply->toolName,
        $reply->toolArgs,
    );

    $messages[] = $reply->asAssistantTurn();
    $messages[] = ['role' => 'tool', 'content' => $result];
}
```

Design lever	Why it matters
Stop conditions	Max steps + a budget cap stop infinite loops and cost blowups.
Tool design	Small, well-described, validated tools = fewer wrong calls.
Context management	Feed only what's relevant; summarise history as it grows.
Error recovery	Return tool errors to the model so it can retry/adapt.
Human-in-the-loop	Gate destructive actions behind approval.
Observability	Trace every step — prompt, tool call, result, cost.

13 · Model Orchestration

WHY IT MATTERS

Real systems use more than one model / call. Orchestration is routing work to the right model and chaining steps reliably.

Pattern	What it means
Routing	Cheap/fast model triages; escalate hard cases to a bigger model.
Prompt chaining	Break a task into steps; each step's output feeds the next.
Parallelisation (fan-out)	Run independent sub-tasks concurrently, then merge.
Orchestrator–worker	A lead model plans and delegates to worker models/agents.
Evaluator–optimiser	One model generates, another critiques and refines in a loop.
Fallbacks	If provider A fails or is rate-limited, fail over to provider B.

Cost/quality trade-off is the real skill: default to the cheapest model that meets the bar, measure with evals, and only spend big-model tokens where it demonstrably helps. Frameworks worth naming: LangChain, LlamaIndex, and lightweight orchestration you roll yourself in PHP/Node when you want full control.

14 · Coding Agents (Claude Code, Codex, OpenCode)

WHY IT MATTERS

A specific responsibility: work with and customize coding agents such as Claude Code, Codex, OpenCode. They want daily-driver fluency plus the ability to extend these tools.

How to customise / extend	Example
Project instructions	A repo-level guide file steering the agent's conventions.
Custom commands / skills	Reusable prompts & workflows for common team tasks.
MCP servers	Model Context Protocol — expose your tools/data to the agent.
Hooks	Run linters/tests/format automatically on the agent's edits.
Sub-agents	Delegate scoped tasks (review, search) to specialised agents.
CI integration	Run agents headless in pipelines for reviews or fixes.

MCP (Model Context Protocol) is the key phrase to know — an open standard for connecting agents to external tools and data sources. Being able to say "I'd expose our internal APIs to the agent via an MCP server so it can act on our own systems safely" is exactly the customisation they're describing.

SAY THIS

"I use a coding agent daily — scaffolding, refactors, test generation — but I review every diff. I'll invest in a good project instruction file and hooks so the whole team gets consistent output."

Watch out

Don't oversell autonomy — they value quality. Frame agents as a force-multiplier you supervise, not a replacement for engineering judgement, tests, or review.

15 · Automation Pipelines

WHY IT MATTERS

Design and implement automation pipelines, and practical experience with at least one automation platform is required. Show you can stitch systems together so work happens without a human clicking buttons.

Building block	Role in a pipeline
Triggers	Webhook, cron/schedule, queue message, file drop, or event.
Queues / workers	Decouple steps; retry failures; scale horizontally.
Idempotency	Safe to re-run a step without double side-effects (see §15).
Dead-letter handling	Park poison messages; alert; replay after a fix.
Orchestration tools	n8n / Zapier / Make for glue; custom code for control.
Observability	Log each stage; alert on failure; track throughput.

Concrete example to have ready: “A webhook fires → job lands on a queue → a worker calls an LLM to classify it → result written to the DB → a Slack/email notification goes out — with retries and a dead-letter queue for failures.” Name whatever platform you’ve actually used (n8n, Zapier, Make, GitLab CI, cron + queues) — one real one beats a list.

16 · Webhooks & Idempotency

WHY IT MATTERS

Integrate SaaS platforms, third-party APIs, and webhooks is a responsibility. Webhooks are how modern integrations stay real-time — and getting idempotency right is what separates a toy from production.

Concern	Handling
Verify authenticity	Check the HMAC signature header against your shared secret.
Respond fast	Return 200 immediately; do the work on a queue.
Idempotency	Dedupe on the event ID — providers retry, so you'll see dupes.
Ordering	Don't assume order; use timestamps/versioning.
Retries	Providers retry on non-2xx; make handlers safe to re-run.
Replay/debug	Log raw payloads; support manual replay.

```
public function handle(Request $request): Response {
    // 1. Verify signature (reject forgeries)
    if (! $this->signatureValid($request)) {
        return response('invalid signature', 401);
    }

    $eventId = $request->input('id');

    // 2. Idempotency: skip events we've already processed
    if (ProcessedEvent::where('event_id', $eventId)->exists()) {
        return response('ok', 200);
    }
    ProcessedEvent::create(['event_id' => $eventId]);

    // 3. Offload work, ack immediately
    ProcessWebhook::dispatch($request->all());
    return response('ok', 200);
}
```

17 · JavaScript / TypeScript

WHY IT MATTERS

Work with JavaScript / TypeScript across the stack — you're not siloed into backend. Show comfort with modern JS and why TypeScript earns its keep.

Topic	What to know
async/await & Promises	Non-blocking I/O; handle errors with try/catch; Promise.all for fan-out.
ES modules	import/export; tree-shaking; bundlers (Vite).
TypeScript types	Interfaces, generics, unions, `unknown` vs `any`, narrowing.
Fetch & streaming	fetch(); read SSE streams for token-by-token LLM output.
Node runtime	Event loop, npm, building CLI tools & API servers.
Immutability & pure fns	Predictable state; easier testing.

```
// Typed, resilient fetch to your PHP backend
interface Article { id: number; title: string; status: 'draft' |
'published'; }

async function getArticles(): Promise<Article[]> {
  const res = await fetch('/v1/articles', {
    headers: { Authorization: `Bearer ${token}` },
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const { data } = (await res.json()) as { data: Article[] };
  return data;
}
```

Why TypeScript: "Types catch a whole class of bugs at compile time, make refactors safe, and act as living documentation — especially valuable in a fast-moving codebase where the team is growing."

18 · React & Vue

WHY IT MATTERS

Frontend frameworks (React, Vue, or similar) are named. You don't need to be a specialist in both — be strong in one and conversant in the shared concepts.

Shared concept	React ⟶ Vue
Components	Function components ⟶ SFCs (.vue).
State	useState ⟶ ref() / reactive().
Side effects	useEffect ⟶ watch() / watchEffect() / lifecycle hooks.
Derived state	useMemo ⟶ computed().
Props & events	props + callbacks ⟶ props + emits.
Reusable logic	Custom hooks ⟶ composables.
Data fetching	React Query / SWR ⟶ Pinia + composables / TanStack Query.

Key talking points: component-driven UI, one-way data flow, the virtual DOM, avoiding unnecessary re-renders (keys, memoisation), and lifting state / a store (Redux, Zustand, Pinia) when it's shared. Tie it back: "I've built the React/Vue front end that consumes the PHP API I wrote — so I own the feature across the stack."

19 · Docker & Containers

WHY IT MATTERS

Docker is in the job title. Responsibilities: set up deployment hosts using Docker and deploy/manage services in containerised environments. Know images vs containers, Dockerfiles, compose, and good hygiene.

Concept	One-liner
Image vs container	Image = frozen blueprint; container = a running instance of it.
Dockerfile	Recipe to build an image, layer by layer (cached).
Multi-stage build	Build in one stage, copy only artefacts to a slim final image.
docker compose	Define multi-service stacks (app + db + redis) in one file.
Volumes vs bind mounts	Persist data / share host files into the container.
Networks	Services talk by name on a shared bridge network.
Env & secrets	Config via env vars; secrets via secret stores, not the image.

```
# Multi-stage: build deps with composer, run on a slim PHP-FPM image
FROM composer:2 AS vendor
WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --optimize-autoloader

FROM php:8.3-fpm-alpine
RUN docker-php-ext-install pdo_mysql opcache
WORKDIR /var/www
COPY --from=vendor /app/vendor ./vendor
COPY . .
EXPOSE 9000
CMD ["php-fpm"]
```

Good hygiene to mention: small base images (Alpine/slim), `.dockerignore`, non-root user, pinned versions, healthchecks, and multi-stage builds to keep images lean. “I keep the final image minimal and reproducible so deploys are fast and the attack surface is small.”

20 · CI/CD with GitLab

WHY IT MATTERS

Automate deployment using GitLab CI/CD pipelines is a named DevOps responsibility, plus general CI/CD and version control (Git) experience. Know the pipeline anatomy and how a commit becomes a deploy.

Concept	Meaning
Pipeline / stages / jobs	Pipeline runs stages in order; jobs in a stage run in parallel.
Runners	Machines/containers that execute jobs.
Artifacts & cache	Pass build output between jobs; cache deps for speed.
Environments	Deploy targets (staging, production) with tracked deploys.
Secrets / variables	Masked CI/CD variables — never hard-code tokens.
Manual gates	`when: manual` to require a click before prod deploy.

```yaml
stages: [test, build, deploy]

test:
  stage: test
  image: php:8.3
  script:
    - composer install
    - ./vendor/bin/phpunit

build:
  stage: build
  script:
    - docker build -t registry.example.com/app:$CI_COMMIT_SHA .
    - docker push registry.example.com/app:$CI_COMMIT_SHA

deploy:prod:
  stage: deploy
  environment: production
  when: manual            # human gate before production
  script:
    - ./deploy.sh $CI_COMMIT_SHA
```

Git fluency assumed: branching strategy (trunk-based or GitFlow), meaningful commits, pull/merge requests with review, and resolving conflicts. CI = catch problems on every push; CD = ship safely and often.

21 · Cloud Deployment

WHY IT MATTERS

Deploy and manage services on cloud platforms and experience with cloud deployments are required. You don't need to be a cloud architect — know the building blocks and how to reason about them.

Building block	What it's for
Compute	VMs, containers (ECS/Cloud Run), or serverless functions.
Managed DB	RDS / Cloud SQL — backups, failover, patching handled.
Object storage	S3 / GCS for files, uploads, static assets, backups.
Load balancer + TLS	Distribute traffic; terminate HTTPS; health checks.
Secrets manager	Store API keys/DB creds outside code & images.
Logs & metrics	Centralised logging + alerting on errors/latency.
IaC	Terraform / compose so infra is reproducible & reviewable.

Frame it pragmatically for a startup: "I'd containerise the app, run it on a managed platform, use a managed Postgres/MySQL, keep secrets in a secrets manager, and wire logging + alerts. Reproducible infra over hand-configured servers." Mention whichever cloud you've actually used (AWS, GCP, DigitalOcean, Fly.io, Hetzner).

22 · Testing & Code Quality

WHY IT MATTERS

Write clean, testable, well-documented code and participate in code reviews. They're opinionated about quality but pragmatic — show you know what to test and when to stop.

Layer	What it covers
Unit tests	One class/function in isolation; mock collaborators. (PHPUnit/Pest)
Integration tests	Real DB/queue; do the parts work together?
Feature / E2E	Full request⟶response; browser flows (Playwright/Cypress).
Static analysis	PHPStan/Psalm catch type errors before runtime.
Linting/formatting	PHP-CS-Fixer, ESLint, Prettier — consistent style, automated.
Testing LLM code	Mock the API; snapshot prompts; use evals for output quality.

Testing non-deterministic AI is a great thing to raise: "I mock the LLM in unit tests for determinism, assert on the prompt I build and how I parse output, and keep a separate eval suite that scores real model responses against a graded dataset." The testing pyramid: many fast unit tests, fewer integration, fewest E2E.

23 · Desirable — Headless CMS & E-commerce

WHY IT MATTERS

Under Desirable Skills: headless CMS, e-commerce platforms, plus open-source contributions or a visible portfolio and AI agent frameworks. Bonus points — mention any you've touched.

Headless CMS

Content managed via API, decoupled from the front end. Names: Strapi (Node), WordPress as headless / WPGraphQL, Contentful, Sanity, Directus. Value: one content source feeds web, app, and AI pipelines.

E-commerce

PHP-heavy space: Magento, WooCommerce, Shopify (APIs/apps), Laravel + Stripe. Talk to: carts, checkout, payment webhooks, idempotent order handling, PCI-safe token handling.

AI agent frameworks to be able to name: LangChain / LangGraph, LlamaIndex, CrewAI, AutoGen, plus MCP for tool integration (see §13). Even "I've read the docs and prototyped with X" shows the curiosity they want. Portfolio: have a GitHub link or a shipped project ready to point at.

24 · The Ideal Candidate — Mindset

WHY IT MATTERS

Section 5 of the JD is pure culture-fit. In a lean startup, how you think matters as much as what you know. These are the exact traits they listed — mirror them with a story for each.

They said they want…	Your proof point
AI as core toolkit, not add-on	A workflow where AI is embedded in how you build, daily.
Opinionated but pragmatic	A time you chose ‘good enough to ship’ over gold-plating.
Gets things shipped	A feature you took from idea to production, fast.
Self-directed, asks good questions	A time you clarified a vague brief before building.
Builds things that work at scale	Something you built that held up under real load.
Excited by ambiguity	A greenfield/uncertain project you drove forward.
Owns end-to-end	Scoping → architecture → delivery → monitoring, all yours.

Prep a STAR story (Situation, Task, Action, Result) for at least three of these. They explicitly value people who see the current AI moment as an opportunity, not a distraction — let genuine enthusiasm show.

25 · Likely Interview Questions

Technical

- Walk me through a PHP feature you shipped end-to-end. Where did the tests and reviews add value?
- How do you integrate an LLM into a production PHP app — auth, retries, cost, structured output?
- Explain an agent loop. How do you stop it running away or costing too much?
- What's prompt injection and how do you defend against it?
- Session-based auth vs JWT — when would you pick each? How do you revoke a JWT?
- You have a slow endpoint backed by a big MySQL table. How do you diagnose and fix it?
- Design a webhook receiver that's safe under retries and duplicate deliveries.
- Multi-stage Dockerfile — why, and what goes in each stage?
- Describe a CI/CD pipeline from commit to production deploy.
- How do you test code that calls a non-deterministic model?

Behavioural / culture

- Tell me about a time you shipped under ambiguity with an incomplete spec.
- How do you decide when 'good enough' is good enough in a startup?
- What's the most interesting thing you've built with AI recently?
- How do you keep up with the fast-moving AI tooling landscape?
- Describe a technical decision you made and how you'd defend it.

> Answer shape: lead with the outcome, then how you got there, then what you'd do differently. Keep it concrete and tie it to their context — a small, fast-moving, AI-native team. Have numbers where you can (latency cut, cost saved, time-to-ship).

26 · Questions to Ask Them

Good questions signal seniority and genuine interest — the JD explicitly values curiosity. Pick a few that fit the conversation.

Product & AI

- Where does AI already live in the product, and where do you want it next?
- Which LLM providers are you on, and how do you manage cost and reliability?
- Are you building agentic workflows in production today? What's worked / broken?
- How do you evaluate model output quality — any eval process yet?

Team & ways of working

- What does the stack look like today, and what's the biggest tech-debt pain?
- How do features get scoped and shipped — what's the path from idea to prod?
- What does success look like for this role in the first 3 and 6 months?
- How is the team using coding agents, and how do you keep quality high?

CLOSING LINE

"This role is exactly where I want to be — PHP fundamentals I'm strong on, plus building real AI features and automation. I like owning things end-to-end and I move fast without dropping quality. I'd love to help shape the engineering culture as the team grows."

Revision guide generated for the AI-Native Full-Stack Developer role (PHP / JavaScript / Docker). Good luck — go and show them AI is already part of how you build.